

1. BookMyShow — Movie Ticket Booking

Interviewer: Let's design BookMyShow — a movie ticket booking platform. Users browse movies and theaters, pick a show, pick seats on a seat map, pay, and get a confirmed ticket. Simple enough on a quiet Tuesday — but design it for a **Friday-night blockbuster release, where thousands of people are staring at the same 200-seat hall, all clicking the same three seats in the front-center row at once.** Walk me through it.

Candidate: This is a fun one because it *looks* like a CRUD app — movies, theaters, shows, seats — but the moment popularity spikes, it becomes a **concurrency-control problem on a shared, tiny resource**: one seat can only ever belong to one buyer, seats are picked interactively (not instantly, like a giveaway), and there's real money involved. Let me clarify before designing.

1. Clarifying the requirements

Candidate: A few questions: 1. Can a user pick **multiple seats** in one booking, and is it **all-or-nothing** — if even one of their chosen seats is taken by someone else a second before they confirm, does the whole booking fail? 2. Once a user selects seats, do we **hold** them while they enter payment, or is seat ownership only decided at the final "Pay" click? 3. Is overselling (two people issued the same seat) an absolute never, or a rare-and-fixable event? 4. Do we need to support seat maps that differ per theater (recliners, gaps, aisles), or is a seat just an ID with a price tier?

Interviewer: Multi-seat, all-or-nothing — you don't want half your group in row F and half in row K. Yes, hold seats the moment they're selected, for a few minutes, so the user can enter payment without someone else stealing the seat mid-checkout. Overselling a seat is a **hard never** — refunding an angry customer and reseating them at a sold-out show is a disaster we plan around from day one. Seat maps vary per theater/screen; assume price tiers (Silver/Gold/Recliner) per row.

Candidate: Then here's what I'm designing for.

Functional - Browse movies → theaters showing it → showtimes → seat map for a specific show. - Select N seats → **hold** them (temporary reservation) → enter payment → **confirm**. - All-or-nothing: if any held seat becomes unavailable before payment completes, the whole booking fails cleanly and the user is told which seat(s) were lost. - Held seats not paid for within a TTL are **released back to the pool** automatically. - Cancel a confirmed booking → seat returns to inventory (subject to policy).

Non-functional — this is where it gets interesting - **Correctness:** a seat is sold to **exactly one** person, ever, for that show. Never two. - **Hot contention on a tiny key space:** a show has maybe 100–300 seats; a blockbuster release means thousands of concurrent users targeting the *same* rows, with a handful of "best" seats hammered hardest — a **hot-key problem at a much smaller scale than the burger giveaway, but with a human, multi-minute checkout window instead of an instant decision.** - **Read-heavy browse path:** movies/theaters/showtimes/seat-map views vastly outnumber actual

bookings (funnel shape) and can tolerate slight staleness. - **Payment safety**: no double charge, no charge-without-a-held-seat. - **Latency**: seat map renders in well under a second; confirm resolves in ~200ms post-payment.

2. Estimation — where the numbers point

5,000 screens, ~6 shows/day/screen, ~200 seats/show → 30,000 shows/day,
~6M seat-slots/day.

Browse vs book funnel: ~2M app opens on release day, mostly the SAME dozen "big" movies, ~20 seat-map views per booking attempt → browse QPS >> booking QPS. 200k bookings/day, half in a 2hr evening spike → 100k/7200s ≈ 14 bookings/sec SYSTEM-WIDE (modest!).

BUT contention concentrates on ONE show at a time. A hot 7pm Friday show, 300 seats, 5,000 people booking in the first 10 minutes:
5,000 / 600s ≈ ~8 seat-hold attempts/sec on THIS ONE SHOW, bursty at open, and many collide on the SAME 10–20 popular seats (center rows). Overall system QPS is low; PER-SHOW, PER-SEAT contention spikes hard for seconds.

⇒ Not a throughput problem like the burger giveaway (millions/sec). It's a CORRECTNESS-under-contention problem on a small, hot key: seats. The design is per-seat locking + hold-with-TTL + all-or-nothing multi-seat semantics, not raw scale.

Candidate: So unlike a flash sale, absolute QPS is manageable. The hard part is **guaranteeing exclusivity on a tiny, hot resource while a human takes 2–5 minutes to pay**, and doing that atomically across *several* seats at once.

3. The naive V1 — and where it breaks

Candidate: Let me sketch the obvious version first: `SELECT status FROM seats WHERE show_id=? AND seat_id IN (...)`, check all AVAILABLE, then `UPDATE ... SET status='HELD'`, in a transaction.

Why this is fragile: between the `SELECT` and the `UPDATE`, **two users can both read "AVAILABLE" for seat A5** and both proceed to mark it held — a classic **read-then-write race**, the same double-booking bug as hotel inventory, just on a smaller, hotter key. Even inside a transaction, a naive `UPDATE ... WHERE status='AVAILABLE'` without explicit row locking can let a second writer "win" a row the first already claimed under weak isolation. We need an explicit, atomic **claim** per seat that also works across *multiple* seats as a unit.

4. The centerpiece — locking seats without overselling

Interviewer: Right — walk me through exactly how you stop two people from both getting seat A5 when they click at the same instant.

Candidate: There are three real options; I'll compare them and pick.

Option A — DB row lock (`SELECT ... FOR UPDATE`) per seat

```
BEGIN;
  SELECT status FROM seats WHERE show_id=? AND seat_id IN ('A5','A6','A7')
  FOR UPDATE;                -- locks exactly these 3 rows
  -- check ALL 3 are AVAILABLE
  UPDATE seats SET status='HELD', held_by=?, hold_expires_at=now()+180s
  WHERE show_id=? AND seat_id IN ('A5','A6','A7');
COMMIT;                      -- lock released
-- a concurrent request touching ANY of these 3 rows blocks until commit,
-- then re-reads and correctly sees HELD -> fails cleanly.
```

Correct and simple: Postgres's row lock is the single source of truth, and locking rows *in a consistent order* (sort seat IDs first) avoids deadlocks when two bookings overlap on some seats. **The problem:** if the lock is held for the whole checkout (not just this instant), it ties up a DB connection for minutes while a human types their card number.

Option B — Redis distributed lock per seat (`SET lock:{showId}:{seatId} NX EX 180`)

```
for seat in [A5, A6, A7]:
  SET lock:{showId}:{seatId} {userId} NX EX 180
if ALL locks acquired -> proceed; if ANY fails -> release the ones we
did acquire (rollback) and tell the user which seat(s) were just taken.
```

Atomic per key, sub-ms, and `EX 180` **is the hold TTL for free** — no sweeper needed to expire a hold. Acquiring multiple keys isn't atomic as a set, so a partial acquire-then-rollback dance is required — but even a crash mid-acquire self-heals via TTL.

Option C — In-process `ReentrantLock` per seat

Only works if one server owns all requests for a show — never true once we scale horizontally behind a load balancer; a lock in pod A's memory doesn't stop a request on pod B. **Rejected.**

My choice: Redis for the fast hold + a DB constraint as the backstop

```
user picks A5,A6,A7 -> REDIS: lock:{showId}:{seatId}=userId (NX, EX 180s)
                        atomic per key, self-expiring, no sweeper needed
                        | all 3 acquired
                        ▼
                        POSTGRES: seats table, UNIQUE(show_id, seat_id),
                        source of truth for CONFIRMED sales
```

Why Redis for the hold, not the DB lock, as the primary mechanism: the hold spans a **human paying** — seconds to minutes — and holding a Postgres row lock across that window ties up a connection and

needs a sweeper. Redis's `EX` makes the hold **self-expiring by construction** — the seat frees itself with zero extra code if the user abandons checkout. **Why still touch Postgres for the final confirm:** money and seat ownership need a durable, ACID-committed record; Redis is memory and can lose state on a bad day. So Redis decides "who gets to *try* to buy this seat right now," and Postgres's `UNIQUE(show_id, seat_id)` constraint on a `CONFIRMED` row is the **final, unforgeable backstop** — even if a bug let two holders reach checkout for the same seat, only one `INSERT / UPDATE` to `CONFIRMED` can win; the second hits a unique-constraint violation and fails loudly rather than silently overselling.

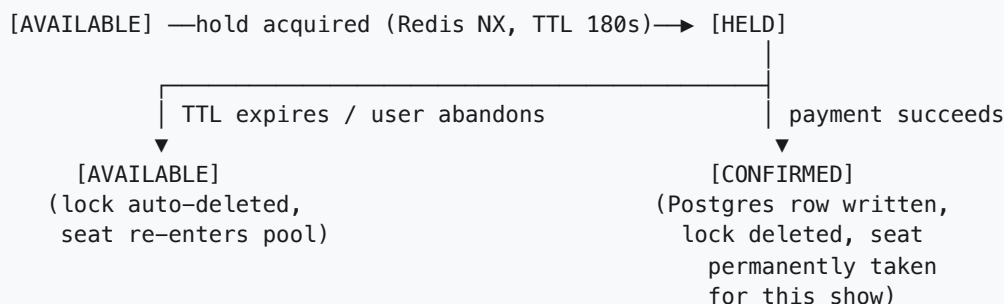
All-or-nothing across multiple seats: acquire locks for **all** requested seats, in **sorted seat-ID order** (deterministic, avoids livelock between overlapping requests); if any single acquire fails, **release whatever we did acquire** and return `SEAT_UNAVAILABLE: [A6]` so the UI can say exactly which seat was lost. Never leave a partial hold hanging:

```
acquireHold(showId, userId, [A5,A6,A7] sorted):
    acquired = []
    for seat in seats:
        if SET lock:{showId}:{seat} userId NX EX 180: acquired.append(seat)
        else: for s in acquired: DEL lock:{showId}:{s}      # roll back and bail
            return FAIL(seat)
    return SUCCESS(acquired)
```

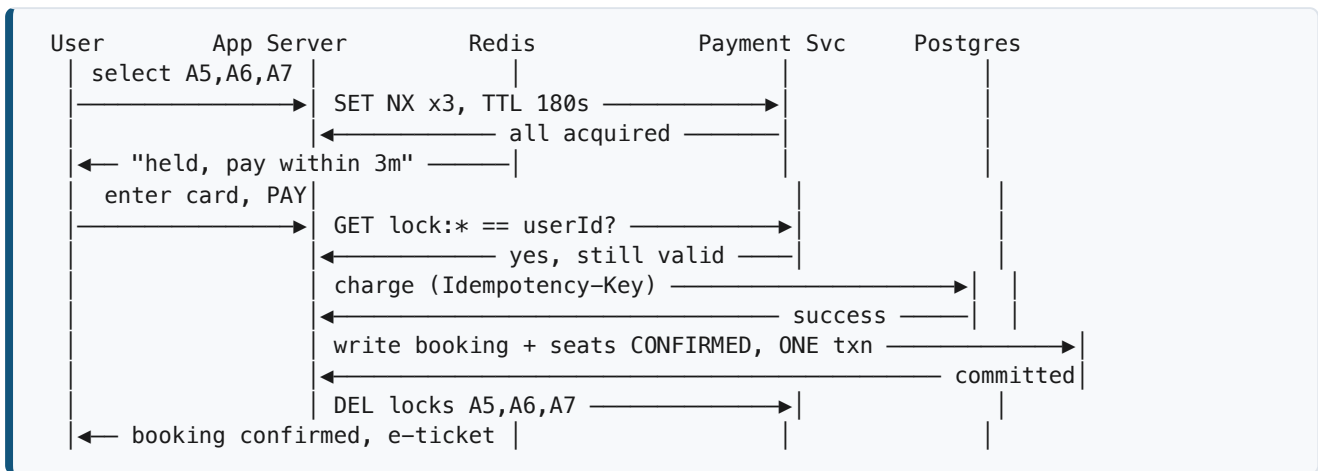
5. The reserve → pay → confirm state machine

Interviewer: Good. Now trace one booking end to end, including payment.

Candidate: Each seat-in-a-show moves through a small state machine; a **booking** ties a group of seats together so they move as one unit.



Sequence for a 3-seat booking:



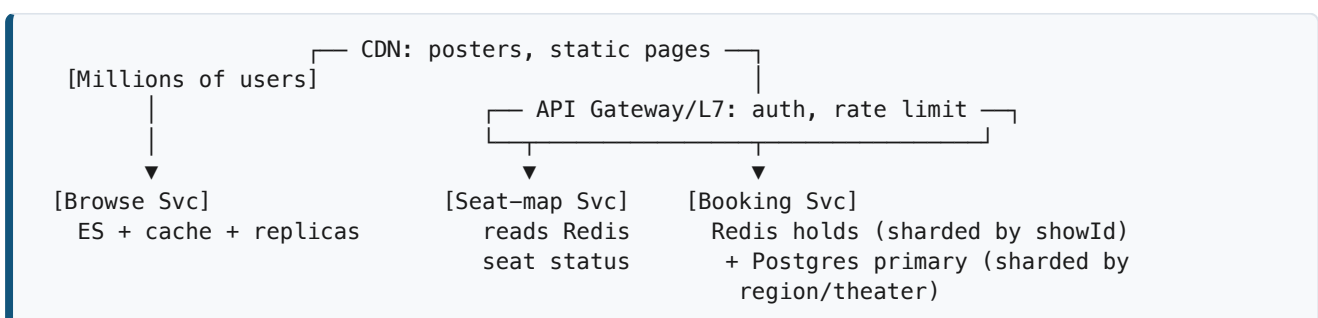
Key correctness step: right before charging, the server **re-validates that the holds are still owned by this user** (not just "does a hold exist" — TTL could have expired and someone else grabbed it a second later). This closes the same gap as the flash sale's "TTL expires while paying" case: **never charge money without re-confirming the hold is still yours at the instant of charge.**

6. Scaling to a nationwide release, then multi-region

Interviewer: Now scale this to a simultaneous nationwide release — 5,000 screens, millions of app opens in one evening. What changes?

Candidate: Two very different problems layer on top of each other:

1. **Browse (movies, theaters, showtimes, seat maps)** is the volume problem — millions of near-identical, highly cacheable reads.
2. **Per-show seat contention** stays *local* — the 5,000 screens don't contend with each other, only users targeting the *same show* do. Sharding by `showId` (all lock/hold traffic for a show hits the same Redis shard) keeps each hot spot small and independent — this is fundamentally a **sharded**, not global, contention problem.



Multi-region: each show has a **home region** (where its theater physically is) — no reason to route a Mumbai theater's booking through a US data center. Bookings write to that region's Postgres primary; the seat-hold Redis shard for that show lives in-region too, so lock latency stays local even though the app is global. Browse traffic serves from the nearest region's cache/replica/index. **Consistency:** the seat lock and `CONFIRMED` write for a given show must be **strongly consistent within that show's home**

region — never eventual, since two regions could otherwise both believe they sold seat A5. A show's authoritative seat state lives in exactly one place, never sharded across regions.

7. Latency — where the milliseconds go

Interviewer: Where's the latency budget on the critical "select seats" click?

Candidate:

Client tap "Hold these seats" → Gateway (auth, ~5ms) → Booking Service
→ Redis SET NX x N seats (sub-ms each, same shard, ~2-5ms total) → response
Total: usually under 30-50ms even at peak — Redis is in-memory and the hot key set is tiny (a few hundred seats/show); no synchronous DB write on the hold path at all (Postgres only touched at final CONFIRM, which is rarer and can tolerate a bit more latency); the seat map itself renders from a cached/denormalized read model, not a live join, so the 200-seat grid is fast too.

Same trick that shows up across all these problems: **push the hot, high-frequency decision (is this seat free right now?) into memory, and keep the slow, must-be-durable step (charge + confirm) off the interactive path** as much as possible while staying synchronous enough that the user sees a real confirmation.

8. Caching — what's safe to be stale, what never is

Candidate: Same principle as hotel inventory: **cache what's for browsing, never cache the number a booking decision depends on.**

SAFE TO CACHE (CDN/Redis cache, seconds-to-minutes TTL): movie details, posters, trailers, theater/showtime lists, and the "seats remaining" BADGE on the browse page ("42 left") — approximate, refreshed on a timer, cosmetic only.

NEVER CACHED, ALWAYS LIVE: the seat MAP's per-seat AVAILABLE/HELD/CONFIRMED status at the instant of clicking "hold" or "pay" — always reads Redis/Postgres directly. A stale "available" seat that's actually taken is a fine UX blemish (user tries it, gets told "just taken, pick another") as long as the ACTUAL hold-acquire step is atomic and authoritative.

Rendering the visual seat *map* can use a slightly-stale snapshot (fast, cached), but the **click-to-hold action itself is always a live, atomic Redis operation** — exactly the split that saved us from oversold hotel rooms: browse is a hint, the write path is the truth.

9. Async — what comes off the critical path

Interviewer: What moves to background workers?

Candidate: Everything after "payment succeeded" that isn't needed to show the user their confirmation:

```
[payment succeeds, booking CONFIRMED in Postgres] emit {bookingId,seats,userId,showId}
▼
[Kafka topic: booking-confirmed] partitioned by showId (per-show ordering)
├─> Notification workers → email/SMS with e-ticket + QR code
├─> Analytics/Bi pipeline → "seats sold" dashboards, revenue reports
├─> Search-index updater → refresh cached "seats remaining" badge
Also on hold EXPIRY (Redis keyspace notification / periodic sweep):
    → emit {showId,seat} released → refresh the cached "seats remaining" badge
```

The user gets "Booking confirmed!" the instant Postgres commits — email, analytics, and the badge refresh happen a moment later, off the critical path, same pattern as the burger/flash- sale audit log.

10. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Seat hold (hot, short-lived lock)	Redis, per-seat key <code>SET NX EX <ttd></code>	Atomic, sub-ms, TTL gives free auto-expiry of abandoned holds. <i>Not ReentrantLock</i> — meaningless across stateless app servers. <i>Not a DB row lock held for the whole checkout</i> — ties up a connection for minutes and needs a sweeper.
Source of truth for confirmed seats/orders	PostgreSQL, <code>UNIQUE(show_id, seat_id)</code> on confirmed rows	ACID across seats + booking + payment; the unique constraint is an unforgeable last-line backstop even if the Redis logic has a bug. <i>Not NoSQL</i> — need multi-row ACID and a real uniqueness guarantee.
Browse/search (movies, theaters, showtimes)	Elasticsearch + CDN/cache	Read-heavy, filter-heavy, tolerant of staleness — fed async from Postgres. <i>Not querying Postgres directly</i> — search traffic would compete with the transactional workload.
Seat-map rendering	Cached snapshot for display, live Redis check on the "hold" click	Rendering doesn't need to be transactionally live; the <i>action</i> does.
Event backbone	Kafka, partitioned by <code>showId</code>	Durable, replayable, ordered-per-show stream for notifications/analytics; decouples slow consumers. <i>Not sync calls</i> — a slow SMS gateway must never delay "Booking confirmed."
Payment	Dedicated Payment Service, idempotent Idempotency-Key	Isolates PCI logic; retries on a flaky network never double-charge.

Concern	Choice	Why this, and why not the alternative
Load balancing / routing	L7 API Gateway	Content-aware routing (browse to a large read fleet, booking/payment to a guarded transactional fleet) plus auth/rate-limit — L4 can't do this.
Sharding key for hot contention	<code>showId</code> , for Redis and Postgres	Confines the worst contention (one blockbuster show) to one shard; unrelated shows never contend.

Say-it-out-loud justification: "Redis gives us a fast, atomic, self-expiring per-seat lock so we can hold a small group of seats for the minutes a human needs to pay, without ever letting two buyers both believe they have the same seat; Postgres is the ACID source of truth with a hard uniqueness constraint as the final backstop against overselling; browse and search are cached and indexed separately because that traffic is huge but tolerant of staleness, while the booking decision itself always reads live state; and sharding everything hot by `showId` keeps one blockbuster's contention from ever touching another screen's inventory."

11. Consistency model — what's strong, what's eventual

- **Strong / linearizable:** seat hold acquisition (Redis atomic `SET NX` per key) and the final `CONFIRMED` write (Postgres transaction + unique constraint). A seat is never held or sold to two people — this is the one guarantee we will not relax.
- **Eventual:** the "seats remaining" badge, the search index, analytics, and notification delivery — all refresh a moment after the authoritative event, explicitly cosmetic.
- **The subtle risk — hold TTL expiring mid-payment:** the server always **re-validates hold ownership immediately before charging**. If the TTL lapsed and the seat was re-grabbed, decline *before* taking money — never charge without a currently-valid hold, mirroring the flash-sale "reservation expired while paying" case.
- **Read-your-writes for the buyer:** the moment you hold a seat, *you* see it held instantly; other users see it free until they next check live — exactly when they'd try to act on it.

12. Failure modes & graceful degradation

Failure	What happens	Mitigation
Redis shard (for a show) dies mid-hold	In-flight holds lost; seats look available again	Replica + failover; worst case a duplicate hold is caught by Postgres's <code>UNIQUE</code> constraint at confirm — never a true oversell
App server crashes mid-acquire	User sees an error; holds still exist with a TTL	TTL auto-releases within minutes even with no cleanup code — self-healing by construction
Payment gateway slow/down	Held seats sit in <code>HELD</code>	TTL protects the seats; user retries within the window; if it lapses, seats release
Postgres primary down	Confirm-writes fail	"Try again shortly"; Redis holds stay valid so the user doesn't lose their seats; failover promotes a replica

Failure	What happens	Mitigation
Two holds race on the same seat, same microsecond	Only one wins	Redis <code>SET NX</code> is atomic per key — no window where both succeed
Search index down	Browse degrades	Booking path unaffected — never depended on search; fall back to a simpler DB-backed listing

13. Follow-up curveballs

Interviewer: A user holds 3 seats, pays, but the app crashes right after payment succeeds and before the booking is marked `CONFIRMED`. What happens?

Candidate: Same "charged but not booked" gap as any payment system. Use a saga/outbox: write a `PENDING_CONFIRM` record (payment intent + held seats) **before** calling the payment gateway; when payment succeeds, flip it to `CONFIRMED` in one transaction. If the app crashes in between, a reconciliation job compares the gateway's transaction log against our booking states and **completes the confirm** (or, if the seats were legitimately re-sold in the gap, refunds and apologizes) — never leave a customer charged with nothing to show for it.

Interviewer: Two friends in different cities are booking seats for the *same* show at the same time, and both want the same 3-seat block. What do they each see?

Candidate: Whoever's `acquireHold` reaches Redis first for a given seat key wins that seat; the loser gets an immediate, specific `SEAT_UNAVAILABLE: [A6]` (not a generic error) so their UI can highlight exactly which seat(s) to change and let them retry with different seats — no full-page failure, no blind retry loop.

Interviewer: How do you stop a user from opening 10 tabs and holding 10 different seat-blocks simultaneously to scalp tickets?

Candidate: Cap **concurrent holds per user per show** — `SET NX` a `user_hold_count:{userId}:{showId}` guard alongside the per-seat locks, capped at the max seats per booking (say 10). Combine with per-account/per-device rate limits at the gateway. Won't stop determined scalpers, but removes the trivial "hold everything in ten tabs" abuse.

Interviewer: Marketing wants a live "X seats left" counter on the show page.

Candidate: Never read the live per-seat Redis keys for this. Compute `remaining = total_seats - count(HELD or CONFIRMED)` on a 1-2s timer (or off the Kafka events), cache it, serve from the CDN — approximate and eventually consistent, exactly right for a cosmetic counter that must never contend with the real seat-lock keys.

14. Deep-dive: "Why not just hold the DB row lock for the whole checkout?"

Interviewer: Redis feels like an extra moving part. Why not just keep the `SELECT ... FOR UPDATE` transaction from §4 **open for the whole 2-5 minutes** a human takes to pay, instead of releasing it and juggling a separate Redis hold? One less system.

Candidate: I get why it's tempting — it's *simpler on paper*, one datastore, one transaction, no TTL bookkeeping. But holding a database lock across **human think-time** is one of the classic ways to take down a database, and it's worth being precise about why.

What actually happens if you hold the lock open

```
BEGIN;
  SELECT status FROM seats WHERE seat_id IN ('A5','A6','A7') FOR UPDATE;
  -- rows now locked --
  -- ... server waits for the USER to type a card number, pass 3-D Secure,
  --    wait on the bank's redirect ... this can take 30s to 5 minutes ...
  UPDATE seats SET status='CONFIRMED' WHERE seat_id IN ('A5','A6','A7');
COMMIT;                                -- lock finally released, minutes later
```

Three separate failure mechanisms stack up here, not just one:

1. **Connection pool exhaustion.** A DB connection (and the transaction on it) is a scarce, stateful resource — typical pools are sized in the tens-to-low-hundreds per app fleet, not thousands. On our hot Friday show (§2: ~8 hold-attempts/sec, each now occupying a connection for up to 180s instead of milliseconds): $8/s * 180s \approx 1,440$ connections needed in flight *just for this one show's checkouts* at steady state. That blows past any sane pool size — new requests (including totally unrelated ones hitting the *same* Postgres instance) start queuing for a connection and timing out.
2. **Lock contention on the exact hot rows.** While seat A5's row is locked, *any* other statement that needs to touch that row — another hold attempt, an admin query, even a read that accidentally takes a lock — blocks behind it. On the row everyone wants (the center-row seat), this serializes exactly the traffic we most need to serve fast.
3. **Deadlocks.** Two overlapping multi-seat bookings — user A locks (A5, A6) while user B concurrently locks (A6, A5) in the opposite order — can each end up waiting on a row the other holds. The DB's deadlock detector eventually kills one transaction, but now that user's multi-minute checkout aborts with an opaque error *after* they've entered payment info, and we're back to a confusing retry.

None of these show up in a demo with two browser tabs. They show up exactly when it matters — the blockbuster spike — because that's when connections, lock queues, and overlapping seat sets all get hot at once.

The comparison

Concern	DB row lock held through checkout (Option A, held minutes)	Redis hold with TTL (our choice)
Resource held during human think-time	A live DB connection + open transaction, for up to the full 2-5 min	One small Redis key (~50 bytes), no connection pinned
Cost of N concurrent holds	N held connections; pool exhaustion once N approaches pool size	N keys in memory; Redis handles millions of keys trivially
What happens if the user abandons checkout	Lock (and connection) sits open until app-level timeout/rollback fires — needs explicit cleanup code	EX 180 expires the key automatically — self-healing, zero cleanup code
Blocking of unrelated traffic	Other statements touching the same row (or, if the pool is exhausted, any row) queue up	Only the exact same key is contended; unrelated seats/shows completely unaffected
Deadlock risk	Real — multiple locked rows held across a long window, classic lock-order deadlock	None — each SET NX is a single-key atomic op, no multi-row lock is ever held open
Crash mid-checkout (server dies)	Transaction eventually rolls back (connection drop), but only after a driver/DB timeout — could hang meaningfully	TTL expires on its own regardless of what the app server does
Right tool when...	The critical section is milliseconds (see below)	The critical section spans human/network time (payment, 3-D Secure redirects, user hesitation)

When a DB lock is the right call

The DB lock isn't wrong in general — it's wrong **for the duration we'd be using it here**. At the final confirm step (§5), we *do* want a short, strict, ACID-guaranteed operation: re-validate the Redis holds, then in one **sub-10ms** transaction, `INSERT / UPDATE` the `CONFIRMED` rows guarded by `UNIQUE(show_id, seat_id)`. That's a lock (implicit, via the unique constraint / row lock) held for **milliseconds around a decision that's already been made** — not minutes around a decision a human is still making. The distinguishing question is always: "*what is this lock waiting on — a fast, deterministic computation, or a slow, unpredictable human/network round trip?*"

Rule of thumb: Hold a DB lock only across work the database itself is doing. The instant a lock's lifetime depends on a human clicking, typing, or waiting on a redirect, move the hold to something that expires on its own — a TTL key, not a transaction.

15. Deep-dive: the hold expires mid-payment — the double-sold-seat race

Interviewer: Walk me through the sharpest failure mode in this design: the seat **hold's TTL expires while the user is sitting on the payment page**, a second user grabs the now-free seat, and *then* the first user's payment succeeds. What actually happens, and how do you stop it?

Candidate: This is the one scenario in the whole design where, without an explicit guard, we could genuinely oversell a seat — so let's trace it exactly.

The race, second by second

```
T+0:00  User A selects A5,A6,A7 -> Redis SET NX lock:{show}:A5..A7 = A, EX 180s
        "Seats held — pay within 3 minutes."
T+0:05  User A reaches the payment page, starts entering card details.
T+1:30  User A's bank throws a 3-D Secure redirect (slow, unpredictable network hop).
T+3:00  TTL EXPIRES. Redis auto-deletes lock:{show}:A5, A6, A7. Seats are AVAILABLE again.
        <-- from this instant, the system believes A5,A6,A7 are up for grabs -->
T+3:05  User B's seat map (or a retry) shows A5,A6,A7 as AVAILABLE.
        User B selects them -> Redis SET NX succeeds (nothing to block it) -> User B
        now holds A5,A6,A7, EX 180s, and proceeds to their own payment.
T+3:40  User A FINALLY completes the 3-D Secure flow. Their card is charged successfully
        at the payment gateway. The app server, having already told User A "you're
        holding these seats," now tries to confirm the booking.

        *** without a re-check, the server would write CONFIRMED for A5,A6,A7 under
        User A's booking — while User B currently believes THEY hold those same
        seats and may be mid-payment or about to confirm too. ***
```

The double-sold-seat risk, made concrete

If the server blindly trusts "I already validated this hold at T+0:00" and confirms on successful payment:

- **If User A's confirm lands first:** Postgres's `UNIQUE(show_id, seat_id)` constraint (§4/§10) will make User B's *subsequent* confirm attempt fail loudly — good, no silent oversell in the *database* — but User B may already be **charged** (or mid-charge) for seats they will never get, which means an angry customer and a refund, exactly the "disaster we plan around from day one" the interviewer flagged in §1.
- **If User B's confirm lands first:** User A's payment has *already succeeded at the gateway* by T+3:40, but their confirm now hits the same unique-constraint failure. We've charged a customer's card for a seat we cannot give them — a charge-without-a-booking bug, the exact failure mode §13 already worries about for crashes, now caused by a TTL race instead.

Either ordering ends the same way: **money moved for a seat that isn't actually theirs**, which is worse than a clean "seat unavailable" rejection — that's recoverable pre-charge, a refund is not free (fees, delay, trust).

Ranked fixes

1. **Extend the hold with a payment-start heartbeat.** When the user reaches the payment page, refresh the TTL (`EXPIRE lock:{show}:{seat} 180`) and/or extend it again right before submitting

to the gateway. Reduces how often this race fires, but doesn't eliminate it — a slow-enough redirect (bank outage, flaky network) can still outlast any fixed extension, and now we're guessing at timeouts instead of proving correctness.

2. **Trust the Postgres unique constraint alone as the guard.** Skip re-checking Redis; just attempt the `CONFIRMED` insert and let the constraint reject the loser. This *does* stop a double-**sold** seat in the database, but only *after* we've already charged the card (as shown above) — it converts a booking bug into a billing bug. Not acceptable when "never oversell" was defined as a hard requirement alongside "no charge without a seat."
3. **Recommended — atomically re-validate hold ownership immediately before capture, and never let a charge happen without it.** Right before calling the payment gateway to **capture** funds (not just before *authorizing* — before actually taking money): `confirmBooking(userId, showId, seats=[A5,A6,A7]): for seat in seats: if GET lock:{showId}:{seat} != userId: return ABORT_BEFORE_CHARGE("seat no longer held", seat) # only if ALL holds are still ours, atomically re-verified, do we proceed: chargeResult = paymentSvc.capture(idempotencyKey=bookingId) if chargeResult != success: return FAIL # single Postgres txn, UNIQUE(show_id, seat_id) as the final backstop INSERT/UPDATE seats,bookings SET CONFIRMED ... DEL lock:{showId}:{seat} for seat in seats` This flips the order of operations so the **hold check gates the charge**, not the other way around. If User A's holds were stolen by User B by T+3:00, User A's confirm attempt at T+3:40 fails the `GET ... == userId` check *before* `paymentSvc.capture()` is ever called — no money moves, and the user is told plainly "your held seats expired, please re-select" instead of being charged and then disappointed.
4. **If we'd already authorized (not yet captured) before this check** — common with card networks that separate auth/capture — void the authorization instead of capturing; nothing settles.
5. **If money has already been captured before we can react** (e.g. a gateway that doesn't support auth/capture separation, or the check itself races with a fire-and-forget capture call), immediately trigger an automated refund as part of the same failure path, and log it for reconciliation — never leave that as a manual, someday task.
6. A weaker fallback some teams use: shorten the TTL and accept the occasional lost hold as a UX cost (user has to re-select), relying on fix #3 to guarantee correctness regardless of TTL length. TTL tuning is a UX knob, not a correctness mechanism — the re-validation-before-charge step is what actually prevents the double sell.

The key insight: **the TTL is allowed to expire — that's it working as designed, freeing an abandoned seat.** The bug isn't the expiry; it's *charging money based on a hold we never re-confirmed still exists*. Fix #3 makes the charge itself conditional on the hold being provably still ours, at the last possible moment, atomically.

What to say out loud: "A TTL expiring mid-payment isn't a bug — it's the hold doing its job of not stranding an abandoned seat forever. The bug would be charging a card without re-proving, atomically, right before the charge, that we still own every seat in the booking. So I gate the payment capture on a fresh `GET lock:{showId}:{seat} == userId` check for all seats — if any hold was lost, we refuse before charging, not after. Postgres's unique constraint is still there as a last-line backstop for the database, but it should never be the thing that catches a bad charge — by the time it fires, the money's already moved."

Summary — the mental model

BookMyShow's booking flow is **not a raw-throughput problem** — it's a "**guarantee exclusive ownership of a tiny, hot set of seats while a human takes minutes to pay, atomically across a multi-seat group**" problem. The winning shape: **per-seat atomic holds in Redis with a TTL** that doubles as automatic hold expiry, acquired **all-or-nothing** across the seats in one booking with deterministic ordering to avoid deadlock/livelock; **Postgres as the ACID source of truth** for confirmed bookings with a hard uniqueness constraint as the unforgeable backstop; **re-validation of the hold immediately before charging** so a TTL race never lets us charge for a seat we no longer hold; and a clean split between the **read-heavy, cacheable browse path** (movies, theaters, seat-map rendering, "seats left" badges) and the **correctness-critical write path** (hold → pay → confirm) that always reads live state and never trusts a cache. Sharding by `showId` keeps one blockbuster's contention from ever touching another screen's seats.